AI Interaction Log

CIE-1, Computer Organisation & Operating Systems | TE IT | Roll No. 33215, Name Adwaiy Deshpande | Assistant used: GitHub Copilot CLI (GPT-5.6 Sol)

I used AI for four limited tasks: interpreting Linux memory columns, predicting the locality result before running it, explaining an unusual CPU topology, and checking process-state terminology. I did not let it supply measurements. Two answers were too confident and one assumed a conventional CPU layout that this laptop does not have. I kept those failures because they show why every hardware-specific statement in the report was checked against command output.

Entry 1

AI WAS WRONG

My prompt

From this `free -h` output, how much memory can a new program use without swapping: 4.0 GiB free, 6.0 GiB buff/cache, and 7.8 GiB available?

AI answer

It added free and buff/cache and said approximately 10 GiB should be usable because cached pages can be discarded. It described the available column as a conservative estimate that excludes cache.

How I checked

I read the `free(1)` manual and compared the answer with the kernel-derived available value printed by the command. I also checked the Swap row before making any claim about what would happen under pressure.

Evidence

```
$ free -h
               total   used   free   shared   buff/cache   available
Mem:            15Gi  7.5Gi  4.0Gi    914Mi        6.0Gi        7.8Gi
Swap:           15Gi     0B   15Gi
```

Conclusion

The AI double-counted reclaimable memory. *Available* already estimates how much can be allocated without swapping and includes the reclaimable part of cache. Not all 6.0 GiB of buff/cache can be reclaimed immediately, so 4.0 + 6.0 is not the right calculation. It also would have been unsafe to discuss swap behaviour without first noticing that 15 GiB of swap is configured and currently unused.

Entry 2

AI WAS OVERCONFIDENT

My prompt

Before I run it, how much slower should column-major traversal be for a 4096 × 4096 array of 32-bit integers on this laptop?

AI answer

It reasoned that a 64-byte cache line holds sixteen integers, so row-major order gets sixteen useful values per line while column-major gets one. It predicted a slowdown near 16× and said hardware prefetching would probably reduce it below that.

How I checked

I compiled with `gcc -O2`, flushed a separate 64 MiB buffer between traversals, alternated which loop ran first, executed seven trials and compared medians rather than selecting one favourable run.

Evidence

```
row-major ms:    13.503 15.371 16.342 21.263 15.142 17.825 14.421
column-major ms: 399.175 439.497 438.782 442.635 428.977 437.953 425.847
median:          15.371 ms versus 437.953 ms = 28.49x
```

Conclusion

The direction and cache-line explanation were correct, but 16× was not a reliable ceiling. The observed slowdown was 28.49×. The simple line-utilisation argument ignores the 16 KiB stride, TLB pressure, reduced prefetch effectiveness and the contiguous loop's greater opportunity for vectorisation. AI turned one component of the cost into a complete prediction.

Entry 3

AI ASSUMED WRONG HARDWARE

My prompt `lscpu` says 8 cores, 2 threads per core and only 12 CPUs. Shouldn't that be 16 logical CPUs?

AI answer It suggested that four logical CPUs were probably disabled in firmware or taken offline by Linux, because it multiplied eight cores by two threads and treated the result as universal.

How I checked I listed the CPU-to-core mapping and checked the processor model. All CPUs were online. Cores 0–3 each had two logical CPUs, while cores 4–7 each had one, which matches the hybrid design of the i5-12450H.

Evidence

```
$ lscpu -e=CPU,CORE,SOCKET,ONLINE
CPU CORE SOCKET ONLINE
  0    0      0    yes       1    0      0    yes
  2    1      0    yes       3    1      0    yes
  4    2      0    yes       5    2      0    yes
  6    3      0    yes       7    3      0    yes
  8    4      0    yes       9    5      0    yes
 10    6      0    yes      11    7      0    yes
```

Conclusion No CPUs were missing. Four performance cores use two hardware threads each and four efficiency cores use one each: 4 × 2 + 4 × 1 = 12. The summary field “threads per core: 2” describes the maximum topology and does not mean every heterogeneous core has two threads. The AI applied a symmetric-core assumption to a hybrid processor.

Entry 4

AI WAS CORRECT

My prompt What do the primary STAT letters I and D mean in `ps`, and why can a D-state process ignore even SIGKILL for a while?

AI answer It said I marks an idle kernel thread and D means uninterruptible sleep, normally while the kernel waits for I/O. SIGKILL remains pending until the kernel operation reaches a safe point because stopping the task midway could leave kernel or device state inconsistent.

How I checked I compared the wording with the PROCESS STATE CODES section of `man ps` and confirmed that the I-state rows on this machine were kernel worker threads.

Evidence

```
$ ps -eo pid,stat,comm | awk '$2 ~ /^I/ {print}' | head -4
    4 I<   kworker/R-rcu_gp
    5 I<   kworker/R-sync_wq
    6 I<   kworker/R-kvfree_rcu_reclaim
    7 I<   kworker/R-slub_flushwq
```

Conclusion The answer matched the manual page and the local evidence. I used the concepts, rewritten in my own words, when describing scheduler states in the report and dashboard.

What I take from this

> The repeated failure mode was not lack of vocabulary; it was treating a general rule as a measurement. AI could explain cache lines, reclaimable memory and simultaneous multithreading fluently, but it could not know this machine's reclaim estimate, timing ratio or hybrid topology until I supplied evidence.
>
> My practical rule is: use AI to identify what to inspect and to explain documented meanings, then verify every machine-specific number with a command, a manual page or an experiment. All numeric claims in the report come from saved output. Where the result disagreed with the prediction, I kept the result and corrected the explanation rather than adjusting the data to fit the theory.